CHAPTER 3

DATA STRUCTURES AND OPERATIONS

CHAPTER ESSENCE AT A GLANCE:
Understand linear and non-linear data structures, stack LIFO push and pop mechanics, queue FIFO enqueue and dequeue with circular buffering, and linked lists.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Linear and Non-Linear Organizations:** Linear structures arrange elements sequentially such that each item possesses a unique predecessor and successor, whereas non-linear hierarchical structures model complex multi-level relationships.

• **Stack and Last-In First-Out Protocol:** A stack restricts all insertions and deletions strictly to the top boundary; inserting into a filled stack causes an overflow condition, while removing from an empty stack triggers an underflow condition.

• **Queue and First-In First-Out Fairness:** A standard queue inserts fresh arrivals at the rear boundary and removes served items from the front; circular wrapping prevents memory fragmentation by recycling freed positions via modulo arithmetic.

• **Linked List Dynamic Flexibility:** A linked list connects dynamically allocated nodes containing data fields and explicit pointer links, enabling efficient insertion and deletion anywhere in memory without shifting adjacent elements.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$\text{Stack Overflow: } \text{top} = \text{MAX} - 1 \quad | \quad \text{Underflow: } \text{top} = -1$

Stack boundary conditions in array implementation

$\text{Queue Empty: } \text{front} = -1 \quad | \quad \text{Queue Full: } \text{rear} = \text{MAX} - 1$

Linear queue boundary conditions (FIFO protocol)

$$\text{rear} = \frac{\text{rear} + 1}{\text{MAX}} \quad (\text{remainder modulo})$$

Circular queue index wrap-around formula

$$\text{newNode} \rightarrow \text{next} = \text{head}; \quad \text{head} = \text{newNode};$$

Singly linked list insertion at beginning in O(1) time

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Always verify top != MAX - 1 before executing push to prevent stack overflow, and top != -1 before pop to prevent underflow.

• **Alert:** In circular queues, calculate next positions using modulo arithmetic (rear + 1) % MAX to recycle available space correctly.

• **Alert:** When deleting nodes in a linked list, update pointer links before freeing memory to avoid memory corruption.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Define data structure. Classify data structures into primitive/non-primitive and linear/non-linear. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Definition):** A specialized format for organizing, processing, retrieving, and storing data in computer memory efficiently. **[1 Mark]**

• **Step 2 (Classification):** Primitive: Directly supported by machine architecture (int, char, float, pointer). Non-Primitive: Derived structures: Linear (elements in sequential order: Arrays, Stacks, Queues, Linked Lists) and Non-Linear (hierarchical/interconnected: Trees, Graphs). **[1 Mark]**

• **Key Exam Result:** Primitive vs Non-primitive (Linear: Array/Stack/Queue; Non-linear: Tree/Graph)

$$\text{Stack Overflow: } \text{top} = \text{MAX} - 1 \quad | \quad \text{Underflow: } \text{top} = -1$$

Stack boundary conditions in array implementation

2. Explain the six basic operations performed on data structures: Traversal, Insertion, Deletion, Searching, Sorting, and Merging. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Six Operations):** 1. Traversal: Accessing every element exactly once. 2. Insertion: Adding a new element at a specified position. 3. Deletion: Removing an existing element. 4. Searching: Finding location of an element with given key (Linear search O(n), Binary search O(log n)). 5. Sorting: Arranging elements in ascending/descending order (Bubble, Selection, Insertion). 6. Merging: Combining two sorted lists into a single sorted list. **[3 Marks]**

• **Key Exam Result:** Traversal, Insertion, Deletion, Searching, Sorting, Merging

$$\text{Average Comparisons} = \frac{N(N-1)}{2} \rightarrow O(N^2)$$

Bubble sort and selection sort worst-case comparisons

3. What is a Stack? Explain the LIFO principle, PUSH, and POP operations with array implementation. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Definition & LIFO):** Stack is a linear data structure following Last-In-First-Out (LIFO) where insertion and deletion occur only at one end called TOP. **[1 Mark]**

• **Step 2 (PUSH Operation):** If top == MAX - 1: Stack Overflow! Else: top++; stack[top] = item; **[1 Mark]**

• **Step 3 (POP Operation):** If top == -1: Stack Underflow! Else: item = stack[top]; top--; return item; **[1 Mark]**

• **Key Exam Result:** PUSH: top++, stack[top]=x | POP: x=stack[top], top--

$$\text{Stack Overflow: } \text{top} = \text{MAX} - 1 \quad | \quad \text{Underflow: } \text{top} = -1$$

Stack boundary conditions in array implementation

4. What is a Queue? Explain the FIFO principle, front and rear pointers, Enqueue and Dequeue operations. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Definition & FIFO):** Queue is a linear data structure following First-In-First-Out (FIFO) where insertion occurs at REAR and deletion at FRONT. **[1 Mark]**

• **Step 2 (Insertion (ENQUEUE)):** If rear == MAX - 1: Queue Overflow! Else: if (front == -1) front = 0; rear++; queue[rear] = item; **[1 Mark]**

• **Step 3 (Deletion (DEQUEUE)):** If front == -1 or front > rear: Queue Underflow! Else: item = queue[front]; front++; if (front > rear) front = rear = -1; **[1 Mark]**

• **Key Exam Result:** Enqueue at REAR; Dequeue from FRONT (FIFO)

$$\text{Queue Empty: } front = -1 \quad | \quad \text{Queue Full: } rear = MAX - 1$$

Linear queue boundary conditions (FIFO protocol)

5. Explain the limitation of a linear queue and how a Circular Queue overcomes it using modulo arithmetic. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Linear Queue Drawback):** In linear array queue, once rear reaches MAX-1, no new elements can be inserted even if empty spaces exist at front due to deletions. **[1 Mark]**

• **Step 2 (Circular Queue Solution):** Last position connects circularly back to first position using modulo arithmetic: rear = (rear + 1) % MAX. Full condition: (rear + 1) % MAX == front. Eliminates wasted memory. **[2 Marks]**

• **Key Exam Result:** Circular increment: index = (index + 1) % MAX

$$\text{Queue Empty: } front = -1 \quad | \quad \text{Queue Full: } rear = MAX - 1$$

Linear queue boundary conditions (FIFO protocol)

6. Explain the structure of a Linked List. Compare arrays and linked lists in terms of memory and performance. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Structure):** Dynamic linear data structure where elements (nodes) are allocated non-contiguously in heap memory. Each node contains: Data field + Next pointer field pointing to subsequent node. **[1 Mark]**

• **Step 2 (Arrays vs Linked Lists):** Array: Fixed size (static), contiguous memory, fast O(1) random access by index, insertion/deletion costly (O(n) shifting). Linked List: Dynamic size, scattered heap memory, sequential access O(n), fast O(1) insertion/deletion without shifting. **[2 Marks]**

• **Key Exam Result:** Linked list: struct Node { int data; Node *next; }; dynamic memory, fast insertion

$$newNode \rightarrow next = head; \quad head = newNode;$$

Singly linked list insertion at beginning in O(1) time

7. Differentiate between a Singly Linked List and a Doubly Linked List with node structure diagrams. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Singly Linked List):** Nodes contain data and single next pointer pointing in forward direction only (unidirectional traversal). **[1 Mark]**

• **Step 2 (Doubly Linked List):** Nodes contain data, prev pointer, and next pointer. Bidirectional traversal (forward and backward). Takes extra memory for pointers. **[1 Mark]**

• **Key Exam Result:** Singly (data + next) vs Doubly (prev + data + next)

$$newNode \rightarrow next = head; \quad head = newNode;$$

Singly linked list insertion at beginning in O(1) time